History of DB:

1.1960s:

Charles Bachman developed the **Integrated Data Store (IDS)** at General Electric, introducing the **network data model**.

IBM created **IMS (Information Management System)**, based on the **hierarchical data model**.

The **SABRE system** was developed for airline reservations, enabling networked data access.

2.1970:

Edgar Codd introduced the **relational data model**, revolutionizing database design and leading to widespread adoption.

3.1980s:

Relational DBMSs became dominant.

SQL became the standard query language.

Advances in **transaction management** enabled efficient and reliable concurrent database access.

4.1990s:

Extensions for new data types (e.g., images, text) and complex queries.

Rise of **data warehouses** and **ERP systems** for enterprise-wide integration.

DBMSs entered the **Internet age**, powering web-based applications.

5.Modern Era:

Focus on **distributed databases**, **multimedia databases**, **data mining**, and **cloud-based systems**.

Enhanced support for **big data** and **scientific projects** like genome mapping.

Reasons of introducing DB:

1.**Data Redundancy Reduction**: Avoid duplication of data, which reduces storage waste and inconsistency across datasets.

2.**Data Consistency**: Ensure updates are reflected across all views, maintaining consistent information.

3.**Data Integrity**: Enforce rules (e.g., constraints) to ensure the accuracy and reliability of data.

4.**Data Security**: Restrict access based on user roles to protect sensitive information.

5.**Data Independence**: Separate data structure and storage from application logic to allow flexible updates without affecting applications.

6.**Concurrent Data Access**: Enable multiple users to access and manipulate data simultaneously without conflict.

7.**Crash Recovery**: Provide mechanisms to restore the database to a consistent state after failures.

8.**Efficient Data Access**: Optimize data retrieval using indexes and query optimization techniques.

9.**Scalability**: Handle growing volumes of data and user requests without performance degradation.

10.**Reduced Application Development Time**: Centralized management of data and built-in tools (like SQL) simplify application development.

11.**Support for Complex Queries and Analysis**: Facilitate sophisticated data querying, reporting, and analysis.

12.**Data Sharing and Collaboration**: Allow multiple departments or organizations to share data efficiently.

Evolution of database systems:

1.1960s: Early Systems

File-Based Systems: Data stored in flat files with no standard structure.

Network and Hierarchical Models:

Network Model: Introduced by Charles Bachman with IDS.

Hierarchical Model: Implemented in IBM's IMS.

Focus on single-user systems and rigid structures.

2.1970s: Relational Model

Edgar Codd introduced the Relational Data Model.

Data organized into tables (relations) with rows and columns.

SQL (Structured Query Language) developed for querying relational databases.

Relational DBMSs gained popularity due to flexibility and ease of use.

3.1980s: Maturity of Relational Systems

Relational DBMSs dominated the market (e.g., Oracle, IBM DB2).

Standardization of SQL (SQL-86 and SQL-89).

Introduction of transaction management for concurrency and recovery.

Focus on performance optimization.

4.1990s: New Features and Applications

Introduction of Object-Oriented Databases (OODBMS).

Emergence of Object-Relational Databases (ORDBMS) combining relational and object-oriented concepts.

Support for multimedia, text, and spatial data.

Rise of Data Warehousing and OLAP for decision support.

Databases integrated with web technologies, enabling dynamic web applications.

5.2000s: Internet and Distributed Databases

Growth of NoSQL Databases for unstructured data (e.g., MongoDB, Cassandra).

Focus on scalability, high availability, and cloud-based databases.

Introduction of distributed databases and Big Data platforms like Hadoop and Spark.

6.Modern Era: Advanced Systems

Advances in NewSQL databases combining scalability of NoSQL with ACID compliance of relational databases.

Integration with AI and Machine Learning.

Emphasis on real-time analytics, graph databases (e.g., Neo4j), and streaming data.

Rise of cloud-native databases and serverless architectures.

Greater focus on data security and privacy regulations (e.g., GDPR).

Various database used as backend:

The 5 most widely used databases (based on popularity, versatility, and adoption across industries) are:

1.MySQL:

Popular open-source RDBMS.

Commonly used in web applications (e.g., WordPress, Facebook).

Known for simplicity and compatibility.

2.PostgreSQL:

Advanced open-source RDBMS with extensibility and robust features.

Supports complex queries, JSON, and large datasets.

Preferred for enterprise and scientific applications.

3.MongoDB:

NoSQL, document-oriented database.

Ideal for unstructured or semi-structured data and real-time applications.

Widely used in modern web and mobile applications.

4.Oracle Database:

Enterprise-grade relational database with advanced features.

Known for reliability, scalability, and performance in large-scale environments.

5.Microsoft SQL Server:

Proprietary RDBMS for enterprise use.

Seamless integration with Windows-based systems and Microsoft tools like Azure.

These databases dominate the industry due to their versatility, scalability, and ability to handle various workloads.

Advantages of DBMS:

1.Data independence: Application programs should be as independent as possible from details of data representation and storage. The DBMS can provide an abstract view of the data to insulate application code from such details.

2.Efficient data access: A DBMS utilizes a variety of sophisticated techniques to store and retrieve data efficiently. This feature is especially important if the data is stored on external storage devices.

3.Data integrity and security: If data is always accessed through the DBMS, the DBMS can enforce integrity constraints on the data. For example, before inserting salary information for an employee, the DBMS can check that the department budget is not exceeded. Also, the DBMS can enforce access controls that govern what data is visible to different classes of users.

4.Data administration: When several users share the data, centralizing the administration of data can offer significant improvements. Experienced professionals who understand the nature of the data being managed, and how different groups of users use it, can be responsible for organizing the data representation to minimize redundancy and for fine-tuning the storage of the data to make retrieval efficient.

5.Concurrent access and crash recovery: A DBMS schedules concurrent accesses to the data in such a manner that users can think of the data as being accessed by only one user at a time. Further, the DBMS protects users from the effects of system failures.

6.Reduced application development time: Clearly, the DBMS supports many important functions that are common to many applications accessing data stored in the DBMS. This, in conjunction with the high-level interface to the data, facilitates quick development of applications. Such applications are also likely to be more robust than applications developed from scratch because many important tasks are handled by the DBMS instead of being implemented by the application.

Database Independence:

1.Data Independence: A major advantage of using a DBMS is data independence, which insulates application programs from changes in data structure and storage.

2.Three Levels of Data Abstraction: Data independence is achieved through three levels of abstraction:

i.External Schema: Defines user views of the data.

ii.Conceptual Schema: Defines the logical structure of the data.

iii.Physical Schema: Defines how data is stored.

3.Logical Data Independence:

Changes in the conceptual schema (e.g., reorganizing relations) do not affect the external schema or user queries.

Example: Splitting a "Faculty" relation into "Faculty public" and "Faculty private" can still support existing queries through modified view definitions.

4.Physical Data Independence:

The conceptual schema hides details of physical storage, such as data layout on disk, file structure, and indexes.

Changes to physical storage do not require changes to the conceptual schema or applications (though performance may vary).

5.Benefits:

Shields users and applications from changes in logical structure (logical data independence).

Insulates users from changes in physical storage (physical data independence).

Extra Info:

Logical data independence: Protects users and applications from changes in the logical organization of the data.

Physical data independence: Shields users and applications from changes in the physical storage details of the data.

DBMS Architecture:

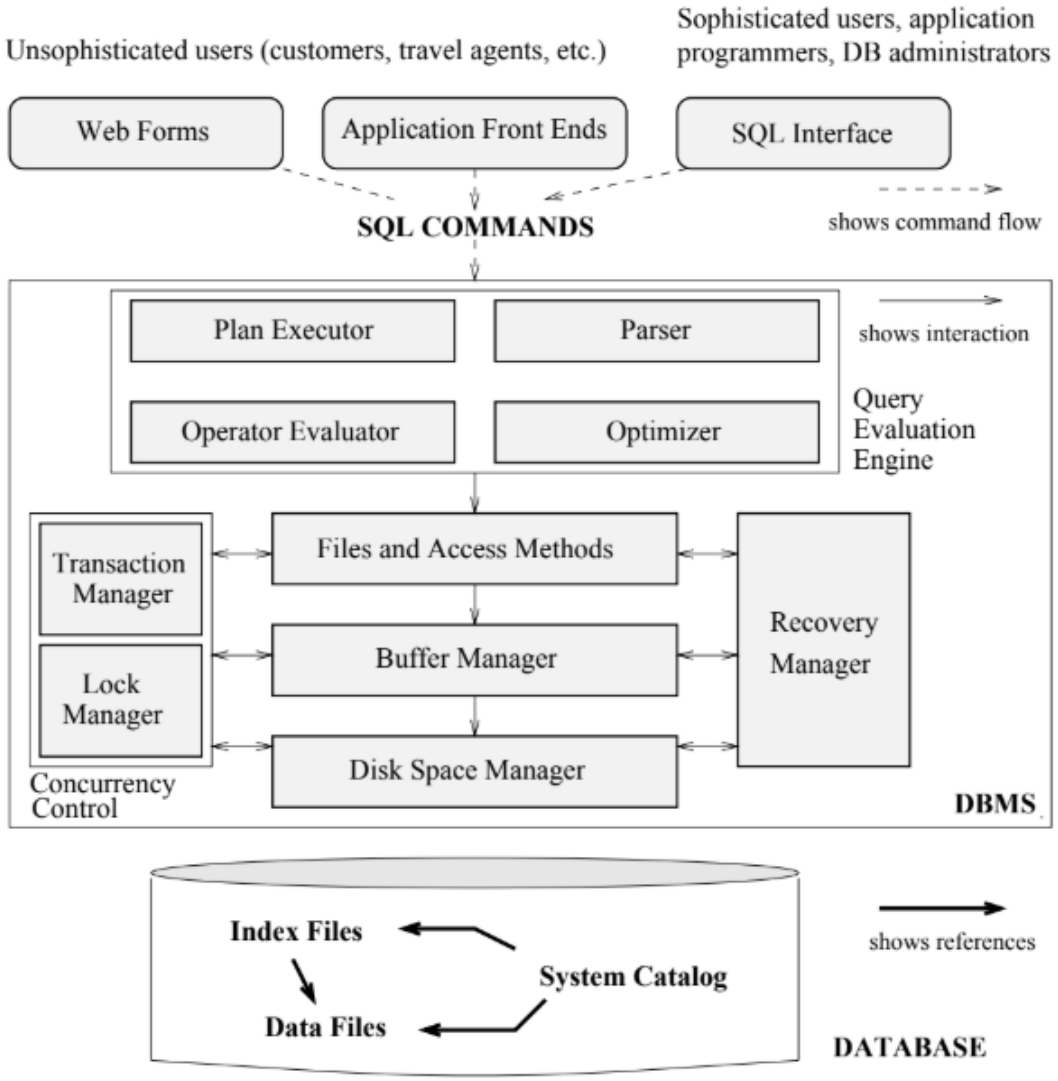


Figure 1.3 Architecture of a DBMS

1.The DBMS accepts SQL commands generated from a variety of user interfaces, produces query evaluation plans, executes these plans against the database, and returns the answers (SQL commands can be embedded in host language application programs, e.g., Java or COBOL programs).

2. When a user issues a query, the parsed query is presented to a query optimizer, which uses information about how the data is stored to produce an efficient execution plan for evaluating the query. An execution plan is a blueprint for evaluating a query, and is usually represented as a tree of relational operators. Relational operators serve as the building blocks for evaluating queries posed against the data.

3. The code that implements relational operators sits on top of the file and access methods layer. This layer includes a variety of software for supporting the concept of a file, which, in a DBMS, is a collection of pages or a collection of records. This layer typically supports a heap file, or file of unordered pages, as well as indexes. In addition to keeping track of the pages in a file, this layer organizes the information within a page.

4. The lowest layer of the DBMS software deals with management of space on disk, where the data is stored. Higher layers allocate, deallocate, read, and write pages through (routines provided by) this layer, called the disk space manager.

5.Concurrency and Crash Recovery: The DBMS manages user requests and maintains a log of all changes to ensure consistency.

Key Components:

i.Transaction Manager: Handles transaction scheduling and lock management using a locking protocol.

ii.Lock Manager: Tracks and grants locks on database objects.

iii.Recovery Manager: Maintains logs and restores the system to a consistent state after crashes.

6.Supporting Components: Disk Space Manager, Buffer Manager, File, and Access Methods: Work in coordination with the above components.

DBMS VS File Systems:

Aspect	**File-Based System**	**DBMS**
Data Storage	Data is stored in flat files managed by the operating system.	Data is stored in a structured format (e.g., tables) managed by the DBMS.
Data Access	Requires custom programs for each query or task.	Provides query languages (e.g., SQL) for efficient and dynamic data retrieval.
Concurrency	Does not handle concurrent access effectively, leading to potential inconsistencies.	Manages concurrent access using built-in mechanisms like locking and transaction management.
Data Security	Provides basic security through operating system passwords.	Offers fine-grained access control and advanced security features (e.g., role-based access).
Crash Recovery	Limited or no support for recovering data after crashes.	Built-in recovery mechanisms ensure data consistency even after failures.

Levels of Abstraction:

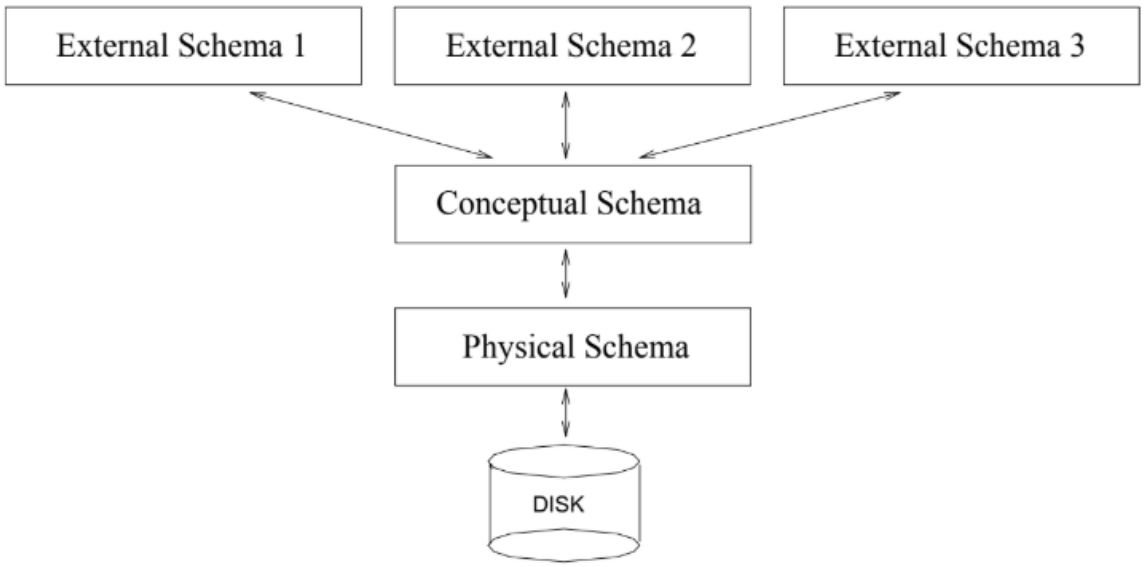


Figure 1.2 Levels of Abstraction in a DBMS

The data in a DBMS is described at three levels of abstraction. The database description consists of a schema at each of these three levels of abstraction: the conceptual, physical, and external schemas. A data definition language (DDL) is used to define the external and conceptual schemas. Information about the conceptual, external, and physical schemas is stored in the system catalogs.

1.Conceptual Schema/Logical Schema:

1.The conceptual schema (sometimes called the logical schema) describes the stored data in terms of the data model of the DBMS. In a relational DBMS, the conceptual schema describes all relations that are stored in the database.

2.Example in Relational DBMS:

- The conceptual schema uses relations (tables) to represent data.
- For example, in a university database:
- Students(sid: string, name: string, login: string, age: integer, gpa: real)
- Faculty(fid: string, fname: string, sal: real)
- Courses(cid: string, cname: string, credits: integer)

3.The choice of relations, and the choice of fields for each relation, is not always obvious, and the process of arriving at a good conceptual schema is called **conceptual database design**.

2.Physical Schema:

1.The physical schema specifies how the data from the conceptual schema is stored on secondary storage (e.g., disks, tapes). We can describes file organization methods used to store the data. Utilizes auxiliary data structures like **indexes** to speed up data retrieval.

2.Store all relations as unsorted files of records. (A file in a DBMS is either a collection of records or a collection of pages, rather than a string of characters as in an operating system.) Create indexes on the first column of the relations such as Students, Faculty, Courses, etc.

3. Decisions about the physical schema are based on an understanding of how the data is typically accessed. The process of arriving at a good physical schema is called physical database design.

3.External Schema:

1.External schemas allow data access to be customized and authorized for individual users or groups. They are represented using the DBMS's data model, like views or relations. Uniqueness of Schemas: A database has **one conceptual schema** and **one physical schema** (for stored relations). It can have **multiple external schemas**, each tailored for specific user needs.

2. A view is conceptually a relation, but the records in a view are not stored in the DBMS. Rather, they are computed using a definition for the view, in terms of relations stored in the DBMS. The external schema design is guided by end user requirements. Records in a view are **computed dynamically** using definitions based on conceptual schema relations.

3.Courseinfo(cid: string, fname: string, enrollment: integer) is an example of a view designed to meet specific user needs (e.g., students finding faculty and enrollment details). External schema design focuses on user requirements and optimizes data access for specific purposes.

It help to avoid redundancy and inconsistency.

Data Models; Data Model is the model that organizes elements of the data and tell how they relate to one-another and with the properties of real-world entities. The basic purpose of the data model is to make sure that the data stored in the data model is understood fully.

Client/Server Architecture:

1. A Client-Server system has one or more client processes and one or more server processes, and a client process can send a query to any one server process. Clients handle user-interface tasks, while servers manage data and execute transactions.

2.Advantages of Client/Server Architecture:

Simplified implementation: The clean separation of functionality and centralized server design makes it easy to implement.

Efficient resource usage: Expensive server machines focus on data management, while inexpensive client machines handle user interactions.

User-friendly: Clients can use familiar graphical user interfaces instead of potentially complex server interfaces.

3. While writing Client-Server applications, maintain a clear **boundary** between client and server functionality. Keep communication **set-oriented** rather than sending many messages (e.g., avoid fetching tuples one at a time).

4. In particular, opening a cursor and fetching one tuple at a time generates excessive messages. Even with client-side caching, advancing the cursor may require additional communication to maintain row locking.

Object Based Logical Model:

1.An object-based logical model in a database management system (DBMS) is a model that uses objects and relationships to represent data. It's a flexible way to structure data and specify constraints.

2.Key Components:

Objects: An **object** is an abstraction of a real-world entity, encapsulating both data and behavior. For example, a STUDENT object might have attributes like Roll_no and Branch, and methods like Setmarks()[1].

Attributes: Attributes describe the properties of an object. For instance, the STUDENT object might have attributes such as Roll_no, Branch, and Marks[1].

Methods: Methods represent the behavior of an object, defining actions that can be performed. For example, the Setmarks() method in the STUDENT object sets the student's marks[1].

Classes: A **class** is a blueprint for creating objects, defining a set of attributes and methods. For example, a Person class might have subclasses like Student, Doctor, and Engineer, each inheriting attributes and methods from the Person class[1].

Inheritance: Inheritance allows new classes to inherit attributes and methods from existing classes, promoting code reusability. For example, the EMPLOYEE class might inherit attributes from the PERSON class.

3.Pros: Flexible: Object-based models provide flexible ways to structure data.

Explicit constraints: Users can explicitly specify constraints on the data.

Cons: Complexity: The model is not fully developed and can be complex to implement.

Adoption: It is not widely accepted by users, partly due to its complexity.

4.Examples of Object-Based Data Models:

i)Entity Relationship (ER) Data Model: The ER model represents real-life scenarios as entities with attributes and relationships. For example, a hospital ER model might include entities like Patient, Doctor, and Tests, each with their respective attributes.

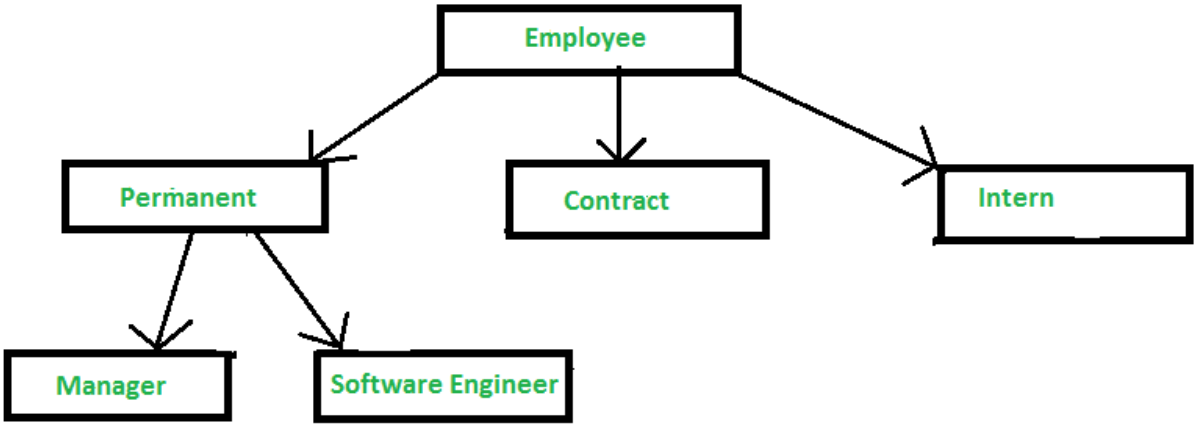
ii)Object-Oriented Data Model: This model represents scenarios as objects, grouping similar objects together and linking them to other objects. For example, the PERSON object might have attributes like Name, Address, and Age, while the EMPLOYEE object inherits these attributes and adds its own.

Record Based Logical Model:

When the database is organized in some fixed format of records of several than the model is known as Record-Based Data Model. It has a fixed number of fields or attributes in each record type and each field is usually of a fixed length.

Further, it is classified into three types:

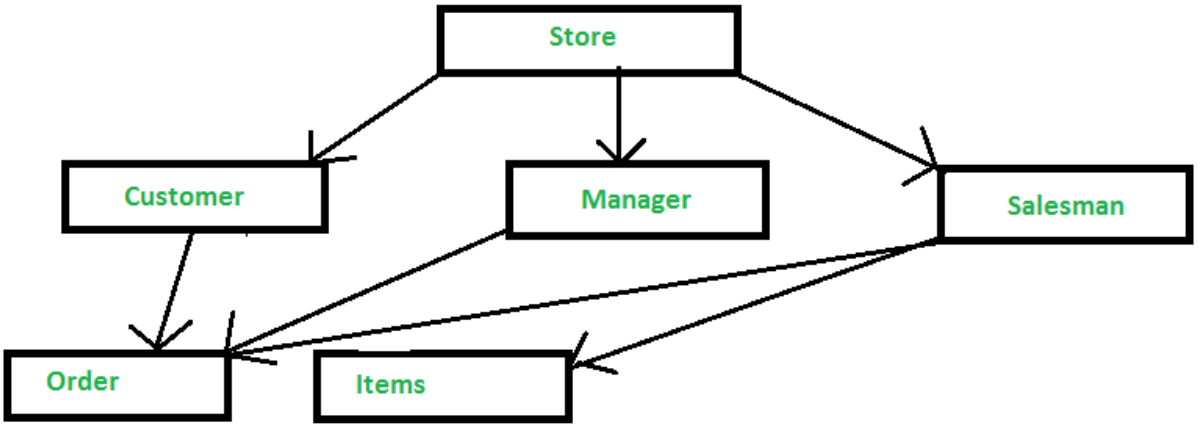
1.Hierarchical Data Model:
In hierarchical type, the model data are represented by collection of records. In this, relationships among the data are represented by links. In this model, tree data structure is used. It was developed in 1960s by IBM, to manage large amount of data for complex manufacturing projects. The basic logic structure of hierarchical data model is upside-down "tree".



Advantages: Simplicity, Data Integrity, Data security, Efficiency, Easy availability of expertise.

Disadvantages: Complexity, Inflexibility, Lack of Data Independence, Lack of querying facility, DML, Lack of standards.

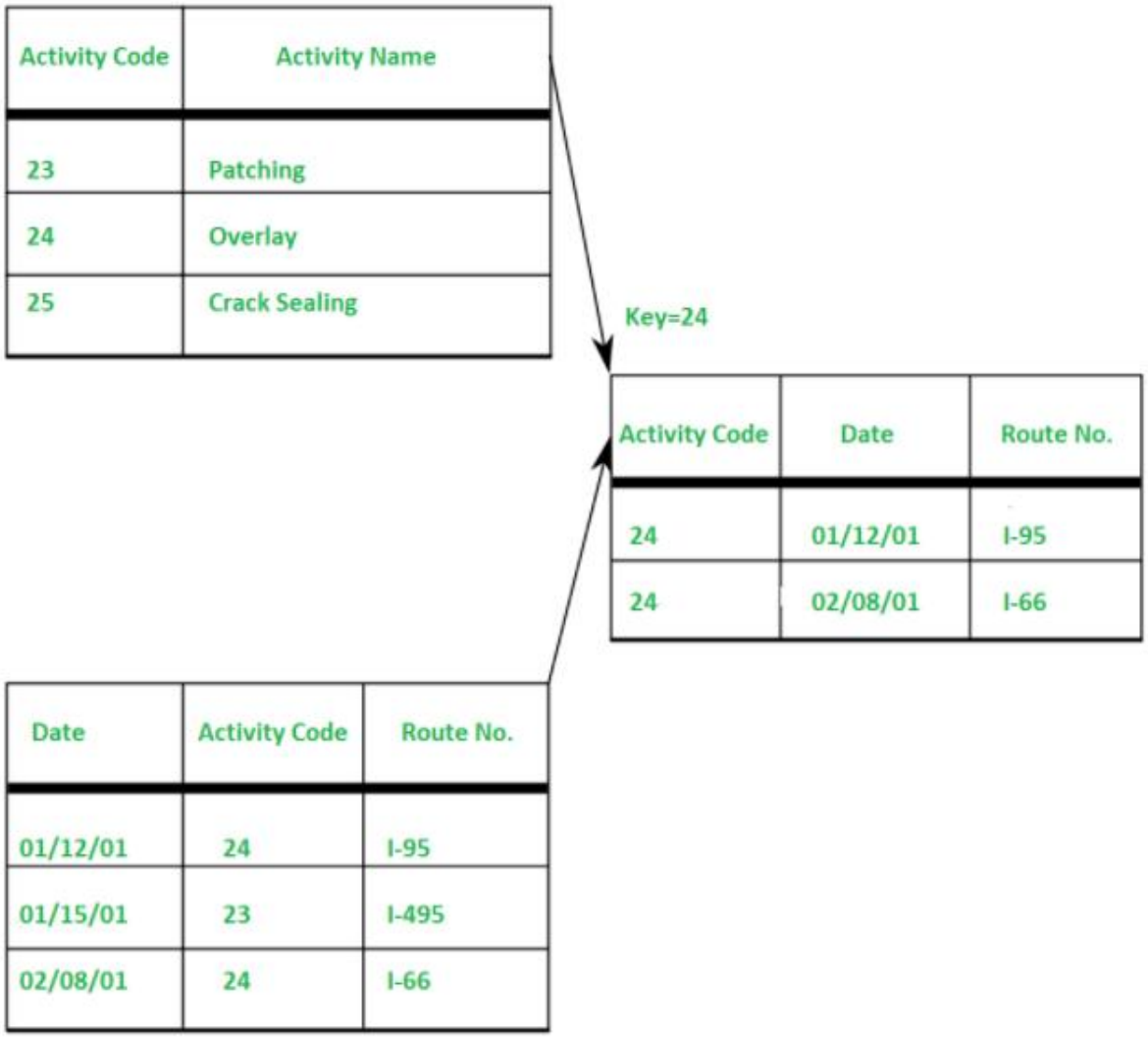
2.Network Data Model:
In network type, the model data are represented by collection of records. In this, relationships among the data are represented by links. Graph data structures are used in this model. It permits a record to have more than one parent. For Example: Social Media sites like Facebook, Instagram etc.



Advantages: Simplicity, Data Integrity, Data Independence, Database standards.

Disadvantages: System Complexity, Lack of structural Independence.

3.Relational Data Model:
Relational Data Model uses tables to represent the data and the relationship among these data. Each table has multiple columns and each column is identified by a unique name. It is a low-level model.



Advantages: Structural Independence, Simplicity, Ease of designing, Implementation, Ad-Hocquery capability.

Disadvantages: Hardware Overheads, Ease of design can result in bad design.

The Relational Model

Relational Constraint.

Key constraint: A key constraint specifies that a particular subset of a relation's fields (known as a candidate key) uniquely identifies a tuple within the relation. Each relation must have at least one key, and out of all candidate keys, one is chosen as the primary key.

Primary key: A primary key in a table that uniquely identifies each row and column or set of columns in the table. The primary key is an attribute or a set of attributes that help to uniquely identify the tuples(records) in the relational table.

Rules For Defining the Primary Key:

1.Minimal: The primary key is composed of a minimum number of attributes that satisfy the unique occurrence of the tuples or records.

2.Accessible: The primary key is used to check the ability to access and interact with the database. The user must easily create, read or delete a tuple using it.

3.NON NULL Value: The primary key which refers to a nonnull value that is required for the identification of the tuple (record) means that any attribute cannot contain the non-null value in DBMS.

4.Time Invariant: The values of the primary key must not change or become null during the lifetime of the relation.

5.Unique: The primary key in which value cannot be duplicated in any of the rows or tuples of the relation.

Ex:

create table student(id number NOT NULL PRIMARY KEY, name varchar(20), course varchar(20), age number(5));

Terminology Related to Primary Key:

1.Candidate Key: A candidate key is any attribute or combination of attributes that uniquely identifies rows in the table and the attribute that forms the key cannot be further reduced.

2.Foreign Key: A foreign key is an attribute of a relationship or group of attributes that could serve as the primary key of another relationship to which it is connected through a relationship.

Ex: create table enrolled(student_id varchar(10), course_id varchar(20), name char(20), PRIMARY KEY (student_id,course_id), FOREIGN KEY (student_id) REFERENCES student);

3.Super key: Super Key is an attribute (or set of attributes) that is used to uniquely identify all attributes in a relation. All super keys can’t be candidate keys but the reverse is true. In relation, a number of super keys is more than a number of candidate keys.

4.Unique Key: The primary key in which value cannot be duplicated in any of the rows or tuples of the Relation table. Although similar to a primary key, a unique key does not necessarily serve as the primary identifier for the table.

Referential Integrity: A referential integrity constraint is also known as **foreign key constraint**. A foreign key is a key whose values are derived from the Primary key of another table. The table from which the values are derived is known as **Master or Referenced** Table and the Table in which values are inserted accordingly is known as **Child or Referencing** Table. In other words, we can say that the table containing the **foreign key** is called the **child table**, and the table containing the **Primary key/candidate key** is called the **referenced or parent table**.

Ex: create table enrolled(student_id varchar(10), course_id varchar(20), name char(20), PRIMARY KEY (student_id,course_id), FOREIGN KEY (student_id) REFERENCES student);

Unique constraint: The UNIQUE constraint ensures that all values in a column are different. Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns. A PRIMARY KEY constraint automatically has a UNIQUE constraint. However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

Ex: CREATE TABLE Persons (ID int NOT NULL UNIQUE, LastName varchar(20) NOT NULL, FirstName varchar(20), Age int);

UNIQUE Constraint on ALTER TABLE:

Ex: ALTER TABLE Persons ADD UNIQUE (ID);

Ex: ALTER TABLE Persons DROP CONSTRAINT UC_Person;

Null constraint: By default, a column can hold NULL values. The NOT NULL constraint enforces a column to NOT accept NULL values. This enforces a field to always contain a value, which means that you cannot insert a new record, or update a record without adding a value to this field.

Ex: CREATE TABLE Persons (ID int NOT NULL, LastName varchar(20) NOT NULL, FirstName varchar(20), Age int);

NULL Constraint on ALTER TABLE:

Ex: ALTER TABLE Persons MODIFY Age int NOT NULL;

Check constraint: The CHECK constraint is used to limit the value range that can be placed in a column. If you define a CHECK constraint on a column it will allow only certain values for this column. If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.

Ex: CREATE TABLE Persons (ID int NOT NULL, LastName varchar(20) NOT NULL, FirstName varchar(20), Age int CHECK(Age>=18));

CHECK Constraint on ALTER TABLE:

Ex: ALTER TABLE PersonsADD CONSTRAINT CHK_PersonAge

CHECK (Age>=18 AND City='Sandnes);

Ex: ALTER TABLE Persons DROP CONSTRAINT CHK_PersonAge;

Enforcing Referential Integrity: Referential integrity is a database design constraint that ensures that foreign keys in a table point to unique primary keys in another table. It helps maintain data quality and prevents data loss.

Options for handling deletes and updates in SQL:

1.NO ACTION (DEFAULT): It rejects the delete/update operation if it violates referential integrity.

2.CASCADE: Deletes/updates all tuples(row) that reference the deleted/updated tuple.

3.SET NULL/SET DEFAULT: Changes the foreign key of the referencing tuple to NULL/ a default value when the referenced tuple is deleted/updated.

Ex: create table enrolled(sid varchar(10), cid varchar(10), grade char(10), PRIMARY KEY (sid), FOREIGN KEY (sid) REFERENCES student ON DELETE CASCADE ON UPDATE NO ACTION);

Ex:

Parent table employees

create table employees(eid number PRIMARY KEY, name varchar(20));

Child table projects

create table projects(pid number, pname varchar(50) eid number, FOREIGN KEY(eid) REFERENCES employees ON DELETE SET NULL ON UPDATE SET DEFAULT);

i.ON DELETE SET NULL: If a referenced eid is deleted from employees, the eid in projects will be set to NULL.

ii.ON UPDATE SET DEFAULT: If a referenced eid is updated in employees, the eid in projects will be set to a default.